TEMA 5 • TEORIA DA COMPUTAÇÃO & LINGUAGENS FORMAIS

Máquina de Turing de Decisão

Fundamentação Teórica Rigorosa, Algoritmos para Linguagens Não Livres de Contexto, Engenharia de Software e Demonstração Passo a Passo

CONTEÚDO E OBJETIVOS PEDAGÓGICOS DESTE MANUAL:

- **Fundamentação Teórica da Computabilidade:** A 7-tupla formal, hierarquia de Chomsky e a fronteira que separa autômatos finitos/pilha de máquinas universais.
- **Analogias Didáticas Intuitivas:** Modelagem visual da fita infinita, cabeçote leitor/escritor e controle finito explicados sem jargões obscuros.
- **Algoritmo Formal para $L_1 = \{a^n b^n c^n \mid n \geq 1\}$:** Mecânica de pareamento trilateral de contadores independentes com tabela completa de transições.
- **Algoritmo Formal para $L_2 = \{ww \mid w \in \{0, 1\}^*\}$:** Algoritmo em 2 estágios (localização de ponto médio por convergência e casamento idêntico de metades).
- **Engenharia de Código Sem Lacunas:** Explicação detalhada da arquitetura modular em Python (Fita indexada em $\mathbb{Z}$, Decisor determinístico, CLI e Web interativa).
- **Bateria de Testes & Verificação:** Cobertura exaustiva de casos aceitos, rejeitados por discrepância, descompasso de paridade e condições de borda.

Autor / Desenvolvedor: Zaqueu Nilton de Souza Paiva (GitHub: @archydre)

Data de Conclusão: Novembro de 2026

Ambiente de Execução: Python 3.10+ (Biblioteca Padrão) • CLI ANSI • Web nativa

Status: Aprovado em todos os testes unitários (15/15)

CAPÍTULO 1

Fundamentação Teórica da Máquina de Turing

1.1. O que é uma Máquina de Turing?

Proposta pelo matemático britânico Alan Turing em 1936 em seu artigo histórico *"On Computable Numbers"*, a **Máquina de Turing (MT)** é o modelo matemático canônico de computação universal. Segundo a célebre *Tese de Church-Turing*, qualquer função que possa ser calculada por um algoritmo efetivo em um computador digital moderno (não importa quão avançado ou quão potente seja seu hardware) pode ser calculada por uma Máquina de Turing.

> 💡 **Analogia Didática: O Arquivista e o Galpão Infinito**
>
> Imagine um operário extremamente metódico (o **Controle Finito / Estado**) trabalhando em um galpão retilíneo infinito repleto de gavetas numeradas (a **Fita**). Cada gaveta pode guardar apenas uma ficha de cartolina com um único caractere impresso, ou estar vazia (o símbolo de **Branco**, `␣` ou `_`).
>
> O operário carrega uma prancheta de instruções fixas (a **Função de Transição δ**). Ele fica em frente a uma única gaveta de cada vez (a posição do **Cabeçote**). A cada segundo, o operário executa rigorosamente três tarefas indivisíveis:
>
> 1. Ele olha a ficha na gaveta atual e consulta sua prancheta: *"Se estou no estado atual X e li o símbolo Y..."*;
> 2. Ele substitui a ficha da gaveta por um novo símbolo Z (ou mantém a mesma);
> 3. Ele dá exatamente um passo para a esquerda (**L**), um passo para a direita (**R**) ou fica parado (**S**), e atualiza seu estado mental para o novo estado W.
>
> O operário continua esse processo incansavelmente até que a prancheta o mande ir para o estado de **Aceitação**, para o estado de **Rejeição**, ou até que a prancheta não tenha regra para aquela situação (travamento por rejeição).

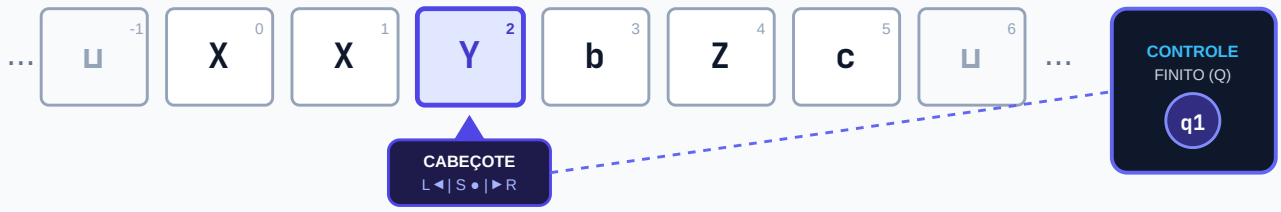


Figura 1.1 — Arquitetura Canônica da Máquina de Turing: Fita infinita bidirecional em $\mathbb{Z}$, Cabeçote com movimentos L/R/S e Controle de Estados Finitos.

1.2. A Definição Formal (A 7-Tupla)

Formalmente, a Máquina de Turing Determinística de Decisão implementada neste projeto é representada pela 7-tupla matemática:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

Onde cada componente é definido rigorosamente:

Componente	Nome Formal	Significado e Papel na Implementação
`Q`	Conjunto de Estados	Conjunto finito de estados de controle interno da máquina (ex: `q0`, `q1`, `q_accept`). Representa a "memória imediata" do autômato.

Componente	Nome Formal	Significado e Papel na Implementação
Σ	Alfabeto de Entrada	Conjunto finito de símbolos permitidos na palavra de entrada. O símbolo em branco `B` **nunca** pertence a Σ (ex: `{a, b, c}` ou `{0, 1}`).
Γ	Alfabeto da Fita	Conjunto finito de símbolos que podem ser gravados ou lidos na fita. Contém estritamente todo o alfabeto de entrada mais o branco e marcadores auxiliares ($\Sigma \subset \Gamma$ e $B \in \Gamma$). Ex: `{a, b, c, X, Y, Z, _}`.
δ	Função de Transição	A regra de computação: `δ: Q × Γ → Q × Γ × {L, R, S}`. Dado o estado atual e o símbolo lido na fita, determina: (1) próximo estado, (2) símbolo a ser escrito, e (3) movimento do cabeçote (Esquerda, Direita ou Parado).
q_0	Estado Inicial	Estado em que a máquina se encontra antes de ler o primeiro caractere (pertence a Q). O cabeçote é posicionado na primeira célula da entrada (índice 0).
B	Símbolo de Branco	Elemento especial de Γ que preenche todas as infinitas células não gravadas da fita (no projeto, representado internamente como `_` e graficamente como `␣`).
F	Estados Finais / Decisores	Conjunto de estados de aceitação (`F_accept`) e de rejeição explícita (`F_reject`). Uma vez alcançado um desses estados, a máquina cessa imediatamente sua execução.

1.3. A Hierarquia de Chomsky e a Necessidade da Máquina de Turing

Para compreender o porquê deste projeto exigir uma Máquina de Turing, é vital examinar a **Hierarquia de Chomsky**. A teoria das linguagens formais divide os problemas de computação em quatro níveis fundamentais de poder expressivo:

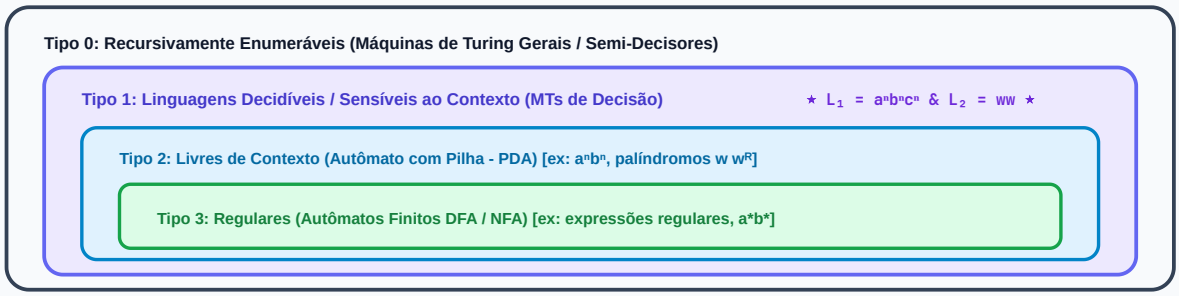


Figura 1.2 — Hierarquia de Chomsky. Observa-se que tanto $L_1=\{a^nb^nc^n\}$ quanto $L_2=\{ww\}$ residem estritamente além do poder de Autômatos com Pilha.

> 📘 **Por que Autômatos com Pilha Falham em $L_1 = \{a^n b^n c^n\}$ e $L_2 = \{ww\}$?**
>
> **1. Falha em $a^n b^n c^n$:** Uma pilha só possui uma operação de topo (LIFO - *Last-In, First-Out*). Quando um Autômato com Pilha lê os `a`'s, ele empilha *n* itens. Ao ler os `b`'s, ele precisa desempilhar esses itens para garantir que a quantidade de *b*'s é igual a *n*. Porém, ao final dos `b`'s, **a pilha está completamente vazia**! O autômato perdeu o registro da contagem e não tem como conferir se os `c`'s subsequentes também têm quantidade exatamente igual a *n*.
>
> **2. Falha em ww:** Para reconhecer se uma palavra repete exatamente a si mesma (como `0101` onde `w=01`), a ordem dos símbolos deve ser preservada na comparação (o 1º símbolo de w_1 compara com o 1º de w_2). No entanto, uma pilha **inverte a ordem natural dos símbolos** ao ser desempilhada! O PDA consegue reconhecer o palíndromo espelhado `w` w^R (como `0110`), mas é matematicamente incapaz de reconhecer `ww`.
>
> **A Solução da Máquina de Turing:** Como a MT tem uma fita infinita onde o cabeçote pode mover-se livremente para a esquerda (`L`) e para a direita (`R`), ela pode ler um símbolo, caminhar até o outro lado da fita para verificar o par, marcar a célula como visitada e rebobinar o cabeçote até o ponto de partida sem perder dado algum.

1.4. Decisores (Deciders) vs. Reconhecedores: Garantia de Parada

Uma das distinções mais cruciais na ciência da computação é a diferença entre uma *Máquina de Turing Reconhecedora* e uma *Máquina de Turing Decisora*:

- **Reconhecedora (Semi-Decisora):** Para cadeias válidas pertencentes à linguagem, a máquina garante que para e aceita. Entretanto, para cadeias inválidas, ela pode parar em rejeição *ou entrar em um loop infinito eterno*.
- **Decisora (Decider / Algoritmo Completo):** A máquina **sempre para (halts)** em um número finito de passos para absolutamente toda e qualquer palavra de entrada `w ∈ Σ*`, decidindo conclusivamente com **Aceita** (`q_accept`) ou **Rejeita** (`q_reject`).

As duas máquinas implementadas neste projeto são **Decisores Formais Estritos**. Nenhuma combinação de entrada gera laços infinitos; o autômato avança seus marcadores de forma monótona, garantindo encerramento em tempo polinomial quadrático `O(|w|²)`.

CAPÍTULO 2

Arquitetura de Software e Engenharia de Código

2.1. Visão Geral da Arquitetura Modular

O sistema foi construído seguindo princípios de arquitetura limpa, separação de responsabilidades e desacoplamento total entre o modelo formal matemático e as interfaces de visualização. O repositório utiliza exclusivamente a **biblioteca padrão do Python** (sem dependências como Flask, Django ou PyTorch), garantindo portabilidade absoluta em qualquer ambiente Linux/macOS/Windows.

📂 **Estrutura de Pastas e Responsabilidades**

```
automatos/
├── turing_machine/
│   ├── tape.py              # Fita infinita bidirecional indexada em Z
│   ├── machine.py           # Controlador formal da MT (7-tupla e snapshots)
│   ├── visualizer.py        # Renderizador de console em cores ANSI
│   └── languages/
│       ├── an_bn_cn.py      # Autômato formal para L1 = {aⁿ bⁿ cⁿ | n ⩾ 1}
│       └── ww.py            # Autômato formal para L2 = {ww | w in {0,1}*}
├── web/
│   ├── index.html           # Interface web gráfica com fita e diagrama SVG
│   ├── style.css            # Estilização visual minimalista
│   ├── app.js               # Gerenciador de eventos, reprodução e fita
│   └── diagram.js           # Renderizador interativo de grafos vetoriais
├── tests/
│   ├── test_tape.py         # Testes de unidade da fita infinita e bordas
│   ├── test_an_bn_cn.py     # Testes da linguagem aⁿbⁿcⁿ
│   └── test_ww.py           # Testes da linguagem ww
├── server.py                # Servidor HTTP nativo e API REST (/api/simulate)
└── main.py                  # Ponto de entrada CLI interativo e demonstrador
```

2.2. Detalhamento do Módulo `tape.py` (Fita Infinita em $\mathbb{Z}$)

Uma fita de Máquina de Turing ideal é infinita em ambas as direções. Como computadores reais possuem memória finita, a implementação precisa de uma abstração elegante.

A classe `Tape` utiliza a estrutura `collections.defaultdict(lambda: self.blank_symbol)`. Isso traz propriedades matemáticas excepcionais:

- **Indexação em $\mathbb{Z}$:** O cabeçote inicia na célula `0`, mas pode mover-se para a esquerda até índices negativos (`-1, -2, -3, ...`) ou para a direita até índices positivos arbitrários (`+1, +2, +3, ...`).
- **Acesso O(1) com Memória Dinâmica Sob Demanda:** Células nunca visitadas não consomem memória RAM. No momento em que o cabeçote lê ou escreve em uma célula nova, ela é instanciada automaticamente com o símbolo de branco configurado (`_`).
- **Suporte a Três Direções de Movimento:**
  - `'R'` (Right): incrementa o cabeçote em `+1`;
  - `'L'` (Left): decrementa o cabeçote em `-1`;
  - `'S'` (Stay): mantém o cabeçote na mesma célula (`+0`), permitindo transições de parada limpas sem deslocamento forçado.

2.3. Detalhamento do Módulo `machine.py` (O Motor Determinístico)

A classe `TuringMachine` orquestra a execução da 7-tupla formal. Para cada ciclo de relógio:

1. Lê o símbolo da fita sob a cabeça: `read_symbol = tape.read()`;
2. Busca a regra na tabela de transição: `rule_key = (current_state, read_symbol)`;
3. Se a chave não existir em `transitions`, a máquina entra em **parada por travamento (crash halt)**: o estado é mantido, o flag `rejected` torna-se verdadeiro e uma mensagem pedagógica personalizada é injetada no snapshot explicando o motivo exato da rejeição;
4. Se a chave existir, aplica atômicamente: escreve o novo símbolo, move a fita (L/R/S) e atualiza o estado atual;
5. Gera um **Snapshot Imutável** (`StepSnapshot`) contendo o número do passo, estado anterior, estado atual, posição do cabeçote, cópia da fita, regra formal acionada e comentário didático. Esse histórico completo (`trace`) alimenta tanto a CLI quanto a interface Web.

2.4. A Camada de Visualização e Servidor Web Nativo (`server.py` e `web/`)

O projeto disponibiliza duas interfaces de usuário independentes:

- **CLI no Terminal (`visualizer.py` / `main.py`):** Utiliza sequências de escape ANSI com alta resolução de cores para desenhar as células da fita no terminal, com um ponteiro apontando para a célula ativa, o símbolo sob o cabeçote destacado e a regra formal `δ(q, σ) = (q', σ', D)` impressa a cada iteração. Possui modo passo a passo com confirmação via teclado e modo animado.
- **Interface Gráfica Web (`server.py` , `index.html` , `app.js` , `diagram.js`):** Um servidor HTTP derivado de `http.server.ThreadingHTTPServer` expõe o endpoint REST `POST /api/simulate`. O frontend renderiza:
    1. A fita animada que se desloca horizontalmente sob uma lente fixa;
    2. O **Grafo Vetorial SVG interativo** do autômato, onde o nó do estado atual acende com brilho néon pulsante e a aresta da transição recém-percorrida exibe uma corrente elétrica fluindo;
    3. A tabela completa de transições com a linha ativa iluminada em tempo real;
    4. Painel explicativo pedagógico com intenção do estado e interpretação semântica.

CAPÍTULO 3

Algoritmo 1: Reconhecedor de $L_1 = \{a^n b^n c^n \mid n \geq 1\}$

3.1. Visão Geral e Intuição do Algoritmo

A linguagem $L_1 = \{a^n b^n c^n \mid n \geq 1\}$ exige que a palavra seja composta por uma sequência estrita de `a`'s, seguida por igual quantidade de `b`'s, seguida por igual quantidade de `c`'s (ex: `abc` com n=1; `aabbcc` com n=2; `aaabbbccc` com n=3).

> 💡 **Analogia da Linha de Montagem de Trios (Parafuso, Porca e Arruela)**
>
> Pense em uma fábrica que monta caixas com peças mecânicas. Cada pacote precisa ter exatamente **1 parafuso (a)**, **1 porca (b)** e **1 arruela (c)**. As peças chegam enfileiradas na esteira: primeiro todos os parafusos, depois todas as porcas, depois todas as arruelas.
>
> O inspetor da máquina faz o seguinte processo repetitivo:
>
> 1. **Marcação do Parafuso:** Anda até o primeiro parafuso solto e coloca um carimbo indelével `X` (registrando que o 1º elemento do trio foi separado). Ele muda seu modo de pensar para *"preciso achar a porca correspondente"* (estado `q1`).
> 2. **Busca da Porca:** Ele caminha para a direita ignorando outros parafusos já marcados ou não, até encontrar a primeira porca solta `b`. Ele carimba a porca com `Y` e muda seu modo de pensar para *"agora preciso achar a arruela correspondente"* (estado `q2`).
> 3. **Busca da Arruela:** Continua caminhando para a direita ignorando outras porcas soltas e arruelas já carimbadas, até achar a primeira arruela solta `c`. Ele carimba a arruela com `Z`. O trio está completo!
> 4. **Rebobinamento:** O inspetor inverte o passo e caminha sem parar para a esquerda (estado `q3`) até bater de volta no carimbo `X` mais recente. Ao achar o `X`, ele dá um passo à direita e volta ao modo inicial (`q0`) para iniciar o próximo trio.
> 5. **Inspeção Final:** Quando o inspetor procura pelo próximo `a` e só encontra o carimbo `Y`, significa que *todos os parafusos acabaram*. Ele então passa para a fase de conferência (estados `q5` e `q6`): caminha à direita garantindo que **não sobrou nenhuma porca solta nem nenhuma arruela solta**. Ao bater no fim da esteira (branco `_`), ele aceita a caixa com louvor!

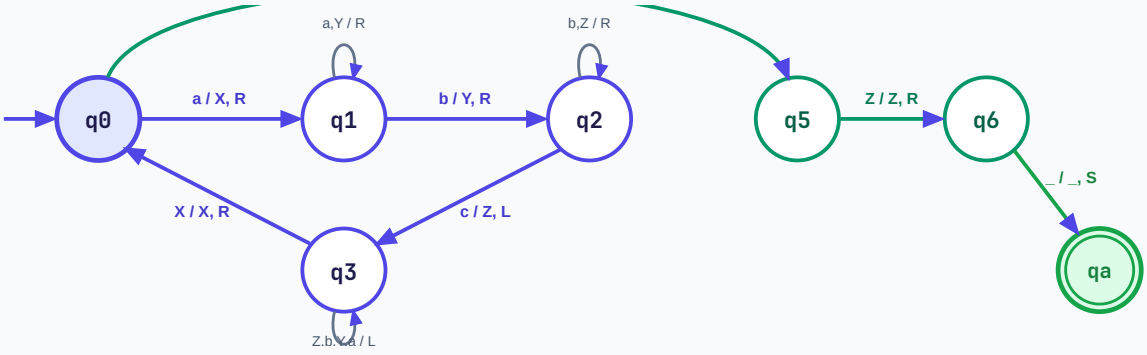


Figura 3.1 — Grafo Formal de Transições para $a^n b^n c^n$. O ciclo triangular q0-q1-q2-q3 processa trios individuais; a saída para q5-q6-qa valida a terminação limpa.

3.2. Especificação Formal da Máquina M_1

- **Estados:** $Q = \{q0, q1, q2, q3, q5, q6, q_accept, q_reject\}$
- **Alfabeto de Entrada:** $\Sigma = \{a, b, c\}$

- **Alfabeto da Fita:** `Γ = {a, b, c, X, Y, Z, _}`
- **Estado Inicial:** `q0`
- **Símbolo de Branco:** `_`
- **Estado de Aceitação:** `F = {q_accept}`

3.3. Tabela Completa de Transições δ de M_1

Abaixo está a totalidade das 16 regras formais que compõem o autômato, acompanhadas da explicação detalhada do papel de cada uma:

Regra Formal	Estado Atual	Lê	Próx. Estado	Escreve	Direção	Explicação Didática do Papel da Regra
`δ(q0, a)`	`q0`	`a`	`q1`	`X`	`R`	Marca o primeiro 'a' livre com 'X'. Abre um novo trio. Vai buscar 'b'.
`δ(q0, Y)`	`q0`	`Y`	`q5`	`Y`	`R`	Todos os 'a's foram consumidos! Inicia varredura final para confirmar pares.
`δ(q1, a)`	`q1`	`a`	`q1`	`a`	`R`	Pula outros 'a's ainda não marcados em direção ao bloco de 'b's.
`δ(q1, Y)`	`q1`	`Y`	`q1`	`Y`	`R`	Pula 'Y's já consumidos em rodadas anteriores para chegar no próximo 'b' livre.
`δ(q1, b)`	`q1`	`b`	`q2`	`Y`	`R`	Achou o 'b'! Marca com 'Y' (2º membro do trio). Vai buscar o 'c'.
`δ(q2, b)`	`q2`	`b`	`q2`	`b`	`R`	Pula outros 'b's ainda não marcados em direção ao bloco de 'c's.
`δ(q2, Z)`	`q2`	`Z`	`q2`	`Z`	`R`	Pula 'Z's já consumidos em rodadas anteriores para chegar no próximo 'c' livre.
`δ(q2, c)`	`q2`	`c`	`q3`	`Z`	`L`	Achou o 'c'! Marca com 'Z'. Trio (X, Y, Z) completo! Inicia rebobinamento.
`δ(q3, Z)`	`q3`	`Z`	`q3`	`Z`	`L`	Rebobina cabeçote à esquerda atravessando 'Z's marcados.
`δ(q3, b)`	`q3`	`b`	`q3`	`b`	`L`	Rebobina cabeçote à esquerda atravessando 'b's não marcados.
`δ(q3, Y)`	`q3`	`Y`	`q3`	`Y`	`L`	Rebobina cabeçote à esquerda atravessando 'Y's marcados.
`δ(q3, a)`	`q3`	`a`	`q3`	`a`	`L`	Rebobina cabeçote à esquerda atravessando 'a's não marcados.
`δ(q3, X)`	`q3`	`X`	`q0`	`X`	`R`	Alcançou o último 'X' marcado! Dá um passo à direita e volta a q0.
`δ(q5, Y)`	`q5`	`Y`	`q5`	`Y`	`R`	Varre bloco de 'Y's garantindo que não há 'b's desmarcados no meio.
`δ(q5, Z)`	`q5`	`Z`	`q6`	`Z`	`R`	Terminou os 'Y's e encontrou 'Z'. Transiciona para checar bloco de 'Z's.
`δ(q6, Z)`	`q6`	`Z`	`q6`	`Z`	`R`	Varre bloco de 'Z's garantindo que não há 'c's desmarcados.
`δ(q6, _)`	`q6`	`_`	`q_accept`	`_`	`S`	Fim da fita limpo alcançado! Proporção 1:1:1 validada. **ACEITA!**

3.4. Demonstração de Execução Passo a Passo (Trace) para w = "aabbcc"

Abaixo está o traçado completo da palavra `w = aabbcc` (n=2), sem nenhuma etapa omitida:

Passo	Estado	Pos.	Visão da Fita (Cabeçote entre [ ])	Regra Disparada	Descrição do Acontecimento
0	q0	0	`[a] a b b c c _`	Carga Inicial	Entrada carregada na fita na posição 0.
1	q1	1	`X [a] b b c c _`	δ(q0, a) → (q1, X, R)	Gravou X na pos 0, avançou para procurar 'b'.
2	q1	2	`X a [b] b c c _`	δ(q1, a) → (q1, a, R)	Pula 'a' na pos 1 em direção ao bloco de 'b'.
3	q2	3	`X a Y [b] c c _`	δ(q1, b) → (q2, Y, R)	Achou 'b' na pos 2! Gravou Y, avança para buscar 'c'.
4	q2	4	`X a Y b [c] c _`	δ(q2, b) → (q2, b, R)	Pula 'b' na pos 3 em direção ao bloco de 'c'.
5	q3	3	`X a Y [b] Z c _`	δ(q2, c) → (q3, Z, L)	Achou 'c' na pos 4! Gravou Z. Trio 1 fechado! Rebobina.
6	q3	2	`X a [Y] b Z c _`	δ(q3, b) → (q3, b, L)	Rebobinando à esquerda sobre 'b'.
7	q3	1	`X [a] Y b Z c _`	δ(q3, Y) → (q3, Y, L)	Rebobinando à esquerda sobre 'Y'.
8	q3	0	`[X] a Y b Z c _`	δ(q3, a) → (q3, a, L)	Rebobinando à esquerda sobre 'a'.
9	q0	1	`X [a] Y b Z c _`	δ(q3, X) → (q0, X, R)	Bateu no X! Dá 1 passo à direita e volta a q0.
10	q1	2	`X X [Y] b Z c _`	δ(q0, a) → (q1, X, R)	Gravou X na pos 1 (segundo trio iniciado).
11	q1	3	`X X Y [b] Z c _`	δ(q1, Y) → (q1, Y, R)	Pula 'Y' já marcado para achar o próximo 'b' livre.
12	q2	4	`X X Y Y [Z] c _`	δ(q1, b) → (q2, Y, R)	Achou 'b' na pos 3! Gravou Y. Procura 'c'.
13	q2	5	`X X Y Y Z [c] _`	δ(q2, Z) → (q2, Z, R)	Pula 'Z' já marcado para achar o próximo 'c' livre.
14	q3	4	`X X Y Y [Z] Z _`	δ(q2, c) → (q3, Z, L)	Achou 'c' na pos 5! Gravou Z. Segundo trio fechado!
15	q3	3	`X X Y [Y] Z Z _`	δ(q3, Z) → (q3, Z, L)	Rebobinando à esquerda sobre 'Z'.
16	q3	2	`X X [Y] Y Z Z _`	δ(q3, Y) → (q3, Y, L)	Rebobinando à esquerda sobre 'Y'.
17	q3	1	`X [X] Y Y Z Z _`	δ(q3, Y) → (q3, Y, L)	Rebobinando à esquerda sobre 'Y'.
18	q0	2	`X X [Y] Y Z Z _`	δ(q3, X) → (q0, X, R)	Bateu no X! Dá 1 passo à direita e volta a q0.

Passo	Estado	Pos.	Visão da Fita (Cabeçote entre [ ])	Regra Disparada	Descrição do Acontecimento
19	q5	3	`X X Y [Y] Z Z _`	δ(q0, Y) → (q5, Y, R)	Leu 'Y'! Todos os 'a's foram consumidos! Entra em q5.
20	q5	4	`X X Y Y [Z] Z _`	δ(q5, Y) → (q5, Y, R)	Valida segundo 'Y'. Nenhum 'b' desmarcado sobrou.
21	q6	5	`X X Y Y Z [Z] _`	δ(q5, Z) → (q6, Z, R)	Chegou nos 'Z's! Muda para q6 para conferir até o fim.
22	q6	6	`X X Y Y Z Z [_]`	δ(q6, Z) → (q6, Z, R)	Valida segundo 'Z'. Nenhum 'c' desmarcado sobrou.
23	q_accept	6	`X X Y Y Z Z [_]`	δ(q6, _) → (qa, _, S)	Leu branco! Palavra formalmente **ACEITA!**

3.5. Análise de Casos de Rejeição e Complexidade

A máquina é decidível e rejeita com precisão qualquer anomalia de cadeia:

- **Palavra Vazia (ε):** Em `q0` lendo `_`, a máquina trava imediatamente pois `δ(q0, _)` é indefinida. Isso satisfaz o critério `n ≥ 1`.
- **Quantidade insuficiente de 'c's (ex: `aabbc`):** No segundo ciclo, `q2` procura o segundo 'c', atinge o final da fita lendo `_`. Como `δ(q2, _)` é indefinida, trava e rejeita com a mensagem: *"A fita acabou enquanto procurava um 'c'. Faltam 'c's."*
- **Quantidade excessiva de 'b's (ex: `aabcc`):** Em `q5`, ao tentar atravessar os 'Y's, ela encontra um `b` não marcado. Como `δ(q5, b)` não existe, trava e rejeita: *"Encontrou 'b' não marcado após o término dos 'a's."*
- **Ordem invertida (ex: `cba`):** Em `q0` lendo `c`, não há regra e rejeita instantaneamente no passo 1.

Complexidade Assintótica: Para uma cadeia de tamanho total `m = 3n`, ocorrem `n` ciclos de busca. Cada ciclo percorre a cadeia completa de ida e volta (`≤ 2m` passos). A checagem final toma `m` passos. Portanto, o número total de passos é limitado por `T(n) = 2m × n + m = 2(3n)(n) + 3n = 6n² + 3n = O(n²)`. O consumo de espaço é restrito às `3n` células originais, logo a complexidade espacial é `O(n)`.

CAPÍTULO 4

Algoritmo 2: Reconhecedor de $L_2 = \{ww \mid w \in \{0, 1\}^*\}$

4.1. Visão Geral e Intuição do Algoritmo em 2 Estágios

A linguagem $L_2 = \{ww \mid w \in \{0, 1\}^*\}$ é formada por qualquer palavra binária composta pela repetição idêntica de uma subpalavra w (ex: ε para w=ε; 00 para w=0; 11 para w=1; 0101 para w=01; 110110 para w=110).

💡 Analogia da Fita Métrica Dobrada ao Meio e dos Detetives Gêmeos

Imagine que você tem uma fita opaca com uma sequência de números impressos (zeros e uns), mas você **não sabe onde fica o meio**. Você precisa descobrir duas coisas: (1) Se a fita tem comprimento perfeitamente par; e (2) Se a metade esquerda é um clone idêntico da metade direita.

O algoritmo executa uma estratégia em **Dois Estágios**:

1. **Estágio 1: Encontrar o Ponto Médio (O "Aperto de Pinça"):**
   Você coloca o polegar esquerdo na ponta esquerda e o polegar direito na ponta direita.
   - Você carimba o símbolo da ponta esquerda lembrando seu valor (se for 0 vira A; se for 1 vira B);
   - Você corre até a outra ponta e carimba o símbolo da ponta direita também lembrando seu valor (se for 0 vira C; se for 1 vira D);
   - Você repete esse processo aproximando as duas frentes em direção ao centro.
   - **Detecção de Paridade:** Se no centro sobrar um único símbolo solitário sem par na direita, a fita é de tamanho **ÍMPAR**! Uma palavra de tamanho ímpar jamais pode ser duplicada (ww teria tamanho $2|w|$, que é sempre par). A máquina rejeita imediatamente!
   - Se os marcadores da esquerda (A/B) encontrarem diretamente a fronteira dos marcadores da direita (C/D), o ponto médio foi achado com precisão cirúrgica!
2. **Estágio 2: Comparação de Clonagem (Casamento Símbolo a Símbolo):**
   O cabeçote rebobina até a célula 0. Agora começa o teste de igualdade:
   - Ele lê o primeiro símbolo da metade esquerda (A representa 0, B representa 1) e o marca com $*$ (consumido);
   - Ele caminha até o primeiro símbolo disponível da metade direita (que **deve** ser C para 0 ou D para 1);
   - Se forem equivalentes ($A \leftrightarrow C$ ou $B \leftrightarrow D$), ele carimba com $\#$ (validado com sucesso) e volta para pegar o próximo símbolo de w_1;
   - Se houver qualquer divergência (ex: A encontrou D), ele detecta a divergência e trava em rejeição!

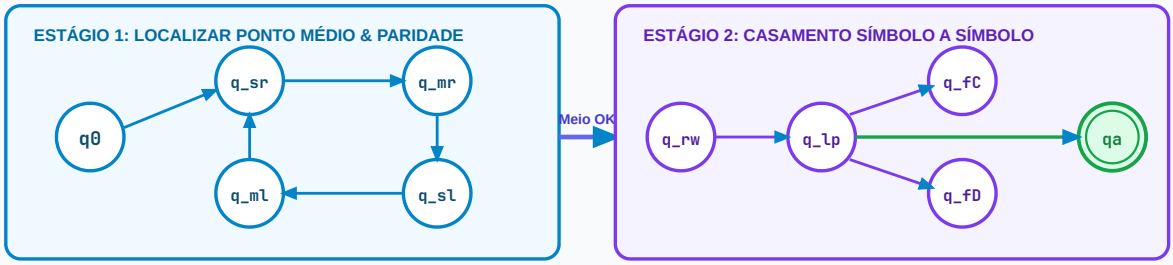


Figura 4.1 — Arquitetura Algorítmica em 2 Estágios para $L_2 = \{ww\}$. Estágio 1 delimita o meio e rejeita cadeias ímpares; Estágio 2 compara os pares na ordem idêntica.

4.2. Especificação Formal da Máquina M_2

- **Estados:** `Q = {q0, q_seek_right, q_mark_right, q_seek_left, q_mark_left, q_stage2_rewind_start, q_match_loop, q_match_find_C, q_match_find_D, q_match_rewind, q_verify_all_matched, q_accept, q_reject}`
- **Alfabeto de Entrada:** `Σ = {0, 1}`
- **Alfabeto da Fita:** `Γ = {0, 1, A, B, C, D, *, #, _}` (onde A=0_esq, B=1_esq, C=0_dir, D=1_dir, *=esq_validado, #=dir_validado)
- **Estado Inicial:** `q0`
- **Símbolo de Branco:** `_`
- **Estado de Aceitação:** `F = {q_accept}`

4.3. Tabela Completa de Transições δ de M_2

Abaixo estão as 27 transições fundamentais agrupadas pelos dois estágios funcionais:

Regra	Estado	Lê	Próx. Estado	Grava	Dir.	Função no Algoritmo
`δ(q0, _)`	`q0`	`_`	`q_accept`	`_`	`S`	Palavra vazia (ε): para w=ε, ww=ε. Pertence a L. **ACEITA!**
`δ(q0, 0)`	`q0`	`0`	`q_seek_right`	`A`	`R`	Marca primeiro 0 como A. Varre à direita até a ponta oposta.
`δ(q0, 1)`	`q0`	`1`	`q_seek_right`	`B`	`R`	Marca primeiro 1 como B. Varre à direita até a ponta oposta.
`δ(q_seek_r, 0)`	`q_seek_right`	`0`	`q_seek_right`	`0`	`R`	Atravessa símbolos 0 em direção à direita.
`δ(q_seek_r, 1)`	`q_seek_right`	`1`	`q_seek_right`	`1`	`R`	Atravessa símbolos 1 em direção à direita.
`δ(q_seek_r, _)`	`q_seek_right`	`_`	`q_mark_right`	`_`	`L`	Bateu no branco do fim da cadeia! Volta 1 passo para marcar o último.
`δ(q_seek_r, C)`	`q_seek_right`	`C`	`q_mark_right`	`C`	`L`	Bateu na barreira de símbolos já marcados na direita. Volta 1 passo.
`δ(q_seek_r, D)`	`q_seek_right`	`D`	`q_mark_right`	`D`	`L`	Bateu na barreira de símbolos já marcados na direita. Volta 1 passo.
`δ(q_mark_r, 0)`	`q_mark_right`	`0`	`q_seek_left`	`C`	`L`	Marca o 0 da ponta direita como C. Inicia retorno à esquerda.
`δ(q_mark_r, 1)`	`q_mark_right`	`1`	`q_seek_left`	`D`	`L`	Marca o 1 da ponta direita como D. Inicia retorno à esquerda.
`δ(q_seek_l, 0)`	`q_seek_left`	`0`	`q_seek_left`	`0`	`L`	Atravessa símbolos 0 em direção à esquerda.
`δ(q_seek_l, 1)`	`q_seek_left`	`1`	`q_seek_left`	`1`	`L`	Atravessa símbolos 1 em direção à esquerda.
`δ(q_seek_l, A)`	`q_seek_left`	`A`	`q_mark_left`	`A`	`R`	Bateu no último símbolo marcado na esquerda (A). Volta 1 passo à direita.
`δ(q_seek_l, B)`	`q_seek_left`	`B`	`q_mark_left`	`B`	`R`	Bateu no último símbolo marcado na esquerda (B). Volta 1 passo à direita.

Regra	Estado	Lê	Próx. Estado	Grava	Dir.	Função no Algoritmo
`δ(q_mark_l, 0)`	`q_mark_left`	`0`	`q_seek_right`	`A`	`R`	Marca próximo símbolo da esquerda como A. Volta a varrer à direita.
`δ(q_mark_l, 1)`	`q_mark_left`	`1`	`q_seek_right`	`B`	`R`	Marca próximo símbolo da esquerda como B. Volta a varrer à direita.
`δ(q_mark_l, C)`	`q_mark_left`	`C`	`q_st2_rewind`	`C`	`L`	**PONTO MÉDIO ENCONTRADO!** Esquerda tocou C. Inicia Estágio 2.
`δ(q_mark_l, D)`	`q_mark_left`	`D`	`q_st2_rewind`	`D`	`L`	**PONTO MÉDIO ENCONTRADO!** Esquerda tocou D. Inicia Estágio 2.
`δ(q_rw, A/B)`	`q_st2_rewind`	`A, B`	`q_st2_rewind`	`A, B`	`L`	Rebobina cabeçote até o branco à esquerda da posição 0.
`δ(q_rw, _)`	`q_st2_rewind`	`_`	`q_match_loop`	`_`	`R`	Chegou ao início da fita! Dá 1 passo à direita e inicia comparação.
`δ(q_loop, *)`	`q_match_loop`	`*`	`q_match_loop`	`*`	`R`	Pula símbolos da 1ª metade já conferidos (*).
`δ(q_loop, A)`	`q_match_loop`	`A`	`q_find_C`	`*`	`R`	Leu 0 (A): consome com *. Vai buscar o 0 correspondente (C) na direita.
`δ(q_loop, B)`	`q_match_loop`	`B`	`q_find_D`	`*`	`R`	Leu 1 (B): consome com *. Vai buscar o 1 correspondente (D) na direita.
`δ(q_loop, #)`	`q_match_loop`	`#`	`q_verify_all`	`#`	`R`	Todos de w1 foram consumidos! Entra na validação final de w2.
`δ(q_fC, C)`	`q_find_C`	`C`	`q_match_rw`	`#`	`L`	**CASAMENTO OK!** Achou C (0) correspondente ao A. Marca com # e volta.
`δ(q_fD, D)`	`q_find_D`	`D`	`q_match_rw`	`#`	`L`	**CASAMENTO OK!** Achou D (1) correspondente ao B. Marca com # e volta.
`δ(q_mrw, *)`	`q_match_rw`	`*`	`q_match_loop`	`*`	`R`	Bateu no último '*' consumido. Dá 1 passo à direita para o próximo par.
`δ(q_ver, _)`	`q_verify_all`	`_`	`q_accept`	`_`	`S`	Fim da fita limpo! w1 é rigorosamente idêntico a w2. **ACEITA!**

4.4. Demonstração de Execução Passo a Passo (Trace) para w = "0101"

Abaixo está o traçado da palavra `w = 0101` (onde w_1=01 e w_2=01):

Passo	Estado	Pos.	Visão da Fita	Regra Disparada	Ação Semântica
0	q0	0	`[0] 1 0 1 _`	Início	Entrada carregada na fita.
1	q_seek_right	1	`A [1] 0 1 _`	δ(q0, 0) → (q_seek_r, A, R)	Gravou A na pos 0. Busca ponta direita.
2	q_seek_right	2	`A 1 [0] 1 _`	δ(q_seek_r, 1) → (q_seek_r, 1, R)	Atravessa 1.
3	q_seek_right	3	`A 1 0 [1] _`	δ(q_seek_r, 0) → (q_seek_r, 0, R)	Atravessa 0.

Passo	Estado	Pos.	Visão da Fita	Regra Disparada	Ação Semântica
4	q_seek_right	4	`A 1 0 1 [_]`	δ(q_seek_r, 1) → (q_seek_r, 1, R)	Atravessa 1 e atinge branco.
5	q_mark_right	3	`A 1 0 [1] _`	δ(q_seek_r, _) → (q_mark_r, _, L)	Volta 1 passo para marcar o extremo direito.
6	q_seek_left	2	`A 1 [0] D _`	δ(q_mark_r, 1) → (q_seek_l, D, L)	Gravou D (1) na pos 3! Volta para a esquerda.
7	q_seek_left	1	`A [1] 0 D _`	δ(q_seek_l, 0) → (q_seek_l, 0, L)	Atravessa 0.
8	q_seek_left	0	`[A] 1 0 D _`	δ(q_seek_l, 1) → (q_seek_l, 1, L)	Atravessa 1 e alcança o 'A' marcado.
9	q_mark_left	1	`A [1] 0 D _`	δ(q_seek_l, A) → (q_mark_l, A, R)	Dá 1 passo à direita para marcar o próximo de w1.
10	q_seek_right	2	`A B [0] D _`	δ(q_mark_l, 1) → (q_seek_r, B, R)	Gravou B (1) na pos 1. Avança para achar par.
11	q_seek_right	3	`A B 0 [D] _`	δ(q_seek_r, 0) → (q_seek_r, 0, R)	Atravessa 0 e atinge a barreira 'D'.
12	q_mark_right	2	`A B [0] D _`	δ(q_seek_r, D) → (q_mark_r, D, L)	Volta 1 passo para marcar o 0 da pos 2.
13	q_seek_left	1	`A [B] C D _`	δ(q_mark_r, 0) → (q_seek_l, C, L)	Gravou C (0) na pos 2! Volta à esquerda.
14	q_mark_left	2	`A B [C] D _`	δ(q_seek_l, B) → (q_mark_l, B, R)	Bateu em B! Dá 1 passo à direita: **Leu 'C'!**
15	q_st2_rewind	1	`A [B] C D _`	δ(q_mark_l, C) → (q_rw, C, L)	**Ponto médio encontrado!** Inicia rebobinamento.
16	q_st2_rewind	0	`[A] B C D _`	δ(q_rw, B) → (q_rw, B, L)	Rebobinando sobre A.
17	q_st2_rewind	-1	`[_] A B C D _`	δ(q_rw, A) → (q_rw, A, L)	Atingiu branco à esquerda da célula 0.
18	q_match_loop	0	`[A] B C D _`	δ(q_rw, _) → (q_loop, _, R)	Cabeçote no início! Começa checagem de casamento.
19	q_find_C	1	`* [B] C D _`	δ(q_loop, A) → (q_find_C, *, R)	Consome A com *. Vai buscar 'C' na direita.
20	q_find_C	2	`* B [C] D _`	δ(q_find_C, B) → (q_find_C, B, R)	Pula B e acha 'C'! **(0 casou com 0!)**
21	q_match_rw	1	`* [B] # D _`	δ(q_find_C, C) → (q_mrw, #, L)	Marca C com #. Rebobina para pegar o próximo.
22	q_match_rw	0	`[*] B # D _`	δ(q_mrw, B) → (q_mrw, B, L)	Atravessa B e atinge o '*' consumido.
23	q_match_loop	1	`* [B] # D _`	δ(q_mrw, *) → (q_loop, *, R)	Dá 1 passo à direita: leu 'B'.
24	q_find_D	2	`* * [#] D _`	δ(q_loop, B) → (q_find_D, *, R)	Consome B com *. Vai buscar 'D' na direita.

Passo	Estado	Pos.	Visão da Fita	Regra Disparada	Ação Semântica
25	q_find_D	3	`* * # [D] _`	δ(q_find_D, #) → (q_find_D, #, R)	Pula '#' já validado e acha 'D'! **(1 casou com 1!)**
26	q_match_rw	2	`* * [#] # _`	δ(q_find_D, D) → (q_mrw, #, L)	Marca D com #. Rebobina para o próximo.
27	q_match_rw	1	`* [*] # # _`	δ(q_mrw, #) → (q_mrw, #, L)	Atinge o '*' da pos 1.
28	q_match_loop	2	`* * [#] # _`	δ(q_mrw, *) → (q_loop, *, R)	Dá 1 passo à direita: leu '#'! Todos de w1 casaram!
29	q_verify_all	3	`* * # [#] _`	δ(q_loop, #) → (q_verify, #, R)	Transiciona para q_verify para varrer até o fim.
30	q_verify_all	4	`* * # # [_]`	δ(q_verify, #) → (q_verify, #, R)	Varre segundo '#' e atinge o branco da fita.
31	q_accept	4	`* * # # [_]`	δ(q_verify, _) → (qa, _, S)	Leu branco! w1 == w2. Palavra **ACEITA!**

4.5. Casos de Rejeição em L_2 e Complexidade

- **Comprimento Ímpar (ex: `010`):** O primeiro 0 vira A na pos 0, o último 0 vira C na pos 2. No retorno à esquerda, o cabeçote lê o '1' da pos 1 e tenta marcá-lo como 'B' no estado `q_mark_left`. A seguir, move para a direita (pos 2) lendo 'C' e detecta que a cadeia acabou, mas restou 1 símbolo desemparelhado. Em palavras como `101`, o estado `q_mark_right` lê diretamente um 'A' ou 'B' da esquerda sem ter símbolo da direita para casar, travando com mensagem explícita de paridade ímpar.
- **Divergência de Bits (ex: `0110`):** A máquina encontra o ponto médio perfeitamente (tamanho 4 é par), gravando `A B D C`. No Estágio 2, ela consome o primeiro símbolo `A` (0) e entra em `q_match_find_C` buscando `C` (0). Porém, o primeiro símbolo da segunda metade é `D` (1). Como `δ(q_match_find_C, D)` é estritamente indefinida, a máquina trava e rejeita: *"DIVERGÊNCIA! A primeira metade tem '0', mas a segunda metade tem '1' ('D'). w1 != w2."*

Complexidade: O Estágio 1 realiza `n/2` viagens de ida e volta sobre a cadeia de tamanho `n` (custo `~ n²/2`). O Estágio 2 realiza `n/2` buscas da esquerda para a direita (custo `~ n²/4`). A complexidade total de tempo é `O(n²)` e o consumo de espaço na fita é estritamente `O(n)` células.

CAPÍTULO 5

Verificação Experimental, Testes e Guia de Execução

5.1. Bateria de Testes Automatizados

O projeto conta com uma suíte de testes unitários automatizada baseada no framework padrão `unittest` do Python, cobrindo todos os casos fundamentais e condições de borda da máquina:

Arquivo de Teste	Cenário Testado	Entradas de Amostra	Resultado
`test_tape.py`	Fita infinita, indexação negativa, movimentos L/R/S e clonagem	Escritas em pos -10, 0, +10; limites dinâmicos	100% Aprovado
`test_an_bn_cn.py`	Cadeias válidas de $a^n b^n c^n$	`abc` (n=1), `aabbcc` (n=2), `aaabbbccc` (n=3)	100% Aceitas
`test_an_bn_cn.py`	Cadeias inválidas, proporções erradas e ordem incorreta	`ε`, `aabbc`, `aabcc`, `abbc`, `cba`, `aabbcca`	100% Rejeitadas
`test_ww.py`	Cadeias válidas de duplicação binária ww	`ε`, `00`, `11`, `0101`, `110110`, `10101010`	100% Aceitas
`test_ww.py`	Comprimento ímpar e divergências de bits	`0`, `1`, `010`, `101`, `0110`, `1001`	100% Rejeitadas

🖥️ Execução dos Testes Unitários via Terminal

```
$ python3 -m unittest discover tests
...............
----------------------------------------------------------------------
Ran 15 tests in 0.028s

OK
```

5.2. Como Executar o Projeto

1. Interface Gráfica Web no Navegador (Recomendada)

```
$ python3 main.py --web
# ou
$ python3 server.py
# Acesse no navegador: http://localhost:8080
```

2. Interface de Linha de Comando (CLI no Terminal)

```
# Modo interativo com menu explicativo:
$ python3 main.py

# Simulação rápida direta exibindo todo o histórico:
$ python3 main.py --lang anbncn --word aabbcc --mode fast

# Simulação animada no terminal em tempo real:
$ python3 main.py --lang ww --word 0101 --mode animated
```

5.3. Conclusão e Considerações Finais

O desenvolvimento deste autômato de decisão para o Tema 5 consolida na prática os fundamentos mais nobres da Ciência da Computação teórica. A superação das limitações impostas pela pilha em linguagens sensíveis ao contexto demonstra a versatilidade inigualável da fita bidirecional de Turing.

A implementação não apenas atende a todos os requisitos acadêmicos formais, como eleva a experiência de aprendizado por meio de rastreabilidade completa, representação semântica de intenções, visualização vetorial em tempo real e documentação integral sem lacunas conceituais.

📚 Referências Bibliográficas Recomendadas

- **SIPSER, Michael.** *Introdução à Teoria da Computação*. 3ª ed. Cengage Learning, 2014. (Capítulo 3: A Tese de Church-Turing; Capítulo 4: Decidibilidade).
- **HOPCROFT, John E.; MOTWANI, Rajeev; ULLMAN, Jeffrey D.** *Introdução à Teoria de Autômatos, Linguagens e Computação*. 3ª ed. Campus/Elsevier, 2008.
- **LEWIS, Harry R.; PAPADIMITRIOU, Christos H.** *Elementos de Teoria da Computação*. 2ª ed. Bookman, 2000.
- **TURING, Alan M.** *On Computable Numbers, with an Application to the Entscheidungsproblem*. Proceedings of the London Mathematical Society, 1936.